

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное образовательное учреждение высшего образования  
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ  
ЗАЩИЩЕН С ОЦЕНКОЙ  
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

\_\_\_\_\_  
должность, уч. степень, звание

\_\_\_\_\_  
подпись, дата

С.Ю. Гуков

\_\_\_\_\_  
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

Телефонная книга

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

\_\_\_\_\_  
подпись, дата

Г.В. Буренков

\_\_\_\_\_  
инициалы, фамилия

Санкт-Петербург 2024

## **1 Цель работы**

Цель данной лабораторной работы заключается в разработке алгоритма, реализующего обход двоичного дерева в трех различных порядках: in-order (внутренний), pre-order (прямой) и post-order (обратный). Данный алгоритм должен быть способен корректно обрабатывать входные данные, представляющие структуру дерева, и обеспечивать последовательный вывод ключей всех узлов в заданных порядках.

В рамках лабораторной работы необходимо реализовать функции построения дерева по заданным входным данным, а также три рекурсивных алгоритма для выполнения обходов дерева. Особое внимание должно быть уделено корректности обработки указателей на левых и правых потомков, исключению ошибок при обработке пустых узлов, а также оптимизации временных затрат на выполнение обходов. Успешное выполнение задачи позволит изучить основные принципы работы с рекурсивными алгоритмами, обработки деревьев и их представления в памяти, а также укрепить навыки разработки программ с использованием структур данных.

## 2 Задание

В данной лабораторной работе требуется разработать программу, которая реализует обход двоичного дерева в трёх порядках: in-order, pre-order и post-order. Входными данными является описание двоичного дерева, содержащее количество узлов и для каждого узла его ключ, индекс левого потомка и индекс правого потомка. Если у узла отсутствуют потомки, это указывается в виде значения -1. Задача заключается в построении дерева на основе входных данных и получении последовательностей ключей всех узлов для каждого из заданных порядков обхода.

Программа должна корректно обрабатывать входные данные, включая случаи, когда у узла отсутствуют потомки, и обеспечивать точность выполнения всех обходов. Алгоритмы для выполнения обходов используют рекурсивные вызовы, чтобы обработать каждый узел дерева в соответствии с заданным порядком. Особое внимание уделяется корректному построению структуры дерева и обработке всех возможных случаев, включая пустые потомки. Программа должна быть устойчива к некорректным входным данным и выдавать предупреждения в случае нарушения формата.

Реализация программы направлена на изучение рекурсивных алгоритмов работы с деревьями и укрепление понимания структуры данных двоичное дерево. Выполнение задачи способствует развитию навыков обработки и проверки входных данных, а также пониманию принципов построения эффективных алгоритмов для работы с деревьями

### **3 Краткое описание хода работы**

В ходе разработки данной лабораторной работы была реализована программа, которая осуществляет обход двоичного дерева в трёх порядках: in-order, pre-order и post-order. Основная задача заключалась в создании алгоритма, способного корректно строить двоичное дерево на основе входных данных и выполнять все три типа обхода, предоставляя результаты в требуемом формате.

Для реализации программы использовалась структура данных, включающая узлы дерева с указателями на левых и правых потомков. Каждый узел дерева описывался ключом и ссылками на потомков. Особое внимание было уделено корректности обработки входных данных, таких как проверка на отсутствие дублирования ссылок, корректность индексов потомков и наличие всех необходимых узлов. Реализованные алгоритмы для обхода дерева использовали рекурсию, что обеспечивало их лаконичность и удобство в реализации.

Программа была разработана с учётом обработки всех возможных ошибок. В случае некорректного ввода, например, указания недопустимых индексов потомков или нарушения структуры дерева, программа фиксировала ошибки и предупреждала пользователя. Также особое внимание уделялось оптимизации кода для обеспечения эффективного выполнения операций даже при больших объёмах данных.

Таким образом, разработанная программа обеспечивает корректное построение дерева, выполнение обходов в заданных порядках и обработку ошибок, что делает её надёжной и эффективной в использовании. Полученные результаты позволяют укрепить знания о работе с деревьями и рекурсивными алгоритмами, а также расширить навыки обработки структурированных данных.

#### 4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
class Node {
    constructor(key, left = null, right = null) {
        this.key = key;
        this.left = left;
        this.right = right;
    }
}

class BinaryTree {
    constructor() {
        this.root = null;
    }

    // Добавление узла
    addNode(key, leftIndex, rightIndex, nodes) {
        let node = nodes[key];
        if (!node) {
            node = new Node(key);
            nodes[key] = node;
        }
        if (leftIndex !== -1) {
            if (!nodes[leftIndex]) {
                nodes[leftIndex] = new Node(leftIndex);
            }
            node.left = nodes[leftIndex];
        }
        if (rightIndex !== -1) {
```

```

        if (!nodes[rightIndex]) {
            nodes[rightIndex] = new Node(rightIndex);
        }
        node.right = nodes[rightIndex];
    }
    return node;
}

```

// Установка корня дерева

```

setRoot(node) {
    this.root = node;
}

```

// In-order обход

```

inOrder(node, result = []) {
    if (node) {
        this.inOrder(node.left, result);
        result.push(node.key);
        this.inOrder(node.right, result);
    }
    return result;
}

```

// Pre-order обход

```

preOrder(node, result = []) {
    if (node) {
        result.push(node.key);
        this.preOrder(node.left, result);
        this.preOrder(node.right, result);
    }
}

```

```

    }
    return result;
}

// Post-order обход
postOrder(node, result = []) {
    if (node) {
        this.postOrder(node.left, result);
        this.postOrder(node.right, result);
        result.push(node.key);
    }
    return result;
}
}

// Чтение входных данных
function parseInput(input) {
    const lines = input.trim().split('\n');
    const n = parseInt(lines[0], 10);
    const nodes = new Array(n).fill(null).map(() => null);
    const tree = new BinaryTree();

    for (let i = 1; i <= n; i++) {
        const [key, left, right] = lines[i].split(' ').map(Number);
        console.log(`Добавление узла: key=${key}, left=${left}, right=${right}`);
        tree.addNode(key, left, right, nodes);
    }

    tree.setRoot(nodes[0]);
    console.log('Дерево успешно построено!');

```

```

    return tree;
}

// Вывод обходов дерева
function printTraversals(tree) {
    console.log('In-order:', tree.inOrder(tree.root).join(' '));
    console.log('Pre-order:', tree.preOrder(tree.root).join(' '));
    console.log('Post-order:', tree.postOrder(tree.root).join(' '));
}

// Пример входных данных
const input = `
5
0 1 2
1 3 4
2 -1 -1
3 -1 -1
4 -1 -1
`;

const tree = parseInput(input);
printTraversals(tree);

```



## 5 Результат работы с примерами

В следующих скриншотах будут предоставлены результаты тестирования программы

```
Добавление узла: key=0, left=1, right=2
Добавление узла: key=1, left=3, right=4
Добавление узла: key=2, left=-1, right=-1
Добавление узла: key=3, left=-1, right=-1
Добавление узла: key=4, left=-1, right=-1
Дерево успешно построено!
In-order: 3 1 4 0 2
Pre-order: 0 1 3 4 2
Post-order: 3 4 1 2 0
```

Рисунок 1 – пример один

```
Добавление узла: key=5, left=1, right=2
Добавление узла: key=1, left=3, right=4
Добавление узла: key=2, left=6, right=-1
Добавление узла: key=3, left=-1, right=-1
Добавление узла: key=4, left=-1, right=-1
Добавление узла: key=6, left=-1, right=-1
Добавление узла: key=0, left=5, right=-1
Дерево успешно построено!
In-order: 3 1 4 5 6 2 0
Pre-order: 0 5 1 3 4 2 6
Post-order: 3 4 1 6 2 5 0
```

Рисунок 2 – пример два

## **6 Выводы**

В результате выполнения лабораторной работы была успешно разработана программа, реализующая обход двоичного дерева в трёх различных порядках: in-order, pre-order и post-order. Программа корректно обрабатывает входные данные, включая описание структуры дерева и информацию о потомках, и обеспечивает точный вывод результатов для каждого типа обхода. Реализация включала обработку всех возможных ошибок, таких как некорректные указания индексов потомков или отсутствие обязательных узлов, что позволило создать надёжное и удобное в использовании решение.

В ходе тестирования программа показала свою работоспособность в различных сценариях: построении деревьев с разной структурой, выполнении обходов, а также обработке случаев, где узлы не имеют потомков. Реализация продемонстрировала устойчивость к некорректным входным данным, включая неверные форматы или нарушение структуры дерева, и корректно сообщала об ошибках. Это свидетельствует о гибкости программы и её универсальности в работе с деревьями.

Полученные результаты подтверждают, что разработанный алгоритм эффективно выполняет поставленные задачи и демонстрирует основные принципы работы с двоичными деревьями. Работа над лабораторной позволила углубить знания о рекурсивных алгоритмах, структуре данных «дерево», а также о важности проверки и валидации данных. Этот опыт способствует развитию навыков программирования и анализа данных, что крайне важно для разработки более сложных алгоритмов и приложений, работающих с деревьями или графами. Программа может быть использована как основа для дальнейших улучшений, таких как реализация других типов обходов, добавление визуализации структуры дерева или анализ времени выполнения операций.